

HAETAE NTT Functional-Correctness Correspondence Catalog

Exhaustive mapping for counted EasyCrypt declarations in `haetae-ntt-verify/easycrypt`

Scope and Counts

This handout gives the mathematician's reading of every counted declaration used in the machine-checked functional-correctness proof for HAETAE's Jasmin NTT. The counted declaration classes are:

$$\#Types = 5, \quad \#Ops = 124, \quad \#Modules = 2, \quad \#Lemmas = 329, \quad \#Hoare/phoare/equiv = 34.$$

The EasyCrypt code column records the exact declaration line as it appears in the source file. For multi-line definitions and proof statements, the displayed line is the source declaration line; the mathematical column gives the precise object, predicate, theorem schema, or semantic judgment denoted by that declaration.

Global Mathematical Notation

Let

$$q = 64513, \quad \mathbb{F}_q = \mathbb{Z}/q\mathbb{Z}, \quad N = 256, \quad \zeta = [426]_q, \quad \text{brev}_8 = 8\text{-bit bit reversal}.$$

A value of EasyCrypt type `coeff Array256.t` is read as a vector $p = (p_0, \dots, p_{255}) \in \mathbb{F}_q^{256}$. A Jasmin byte array rp is decoded by

$$D(rp)_i = [\text{to_sint}(\text{get32}(rp, i))]_q, \quad 0 \leq i < 256.$$

The final forward and inverse specifications are

$$F(p)_i = \sum_{j=0}^{255} p_j \zeta^{(2 \text{brev}_8(i)+1)j},$$
$$F^{-1}(p)_i = [256]_q^{-1} \sum_{j=0}^{255} p_j \zeta^{-(2 \text{brev}_8(j)+1)i}.$$

The proof uses the forward partial sums

$$P_{\ell,s,b}(p) = \sum_{u=0}^{\ell-1} \zeta^{(2s+1) \text{brev}_8(u)} p_{\text{brev}_8(b\ell+u)}$$

and inverse partial sums

$$Q_{\ell,s,b}(p) = \sum_{u=0}^{\ell-1} p_{b\ell+u} \zeta^{-(2 \text{brev}_8(b\ell+u)+1)s}.$$

The Hoare-style semantic readings are

$$\{P\} c \{Q\} \quad \equiv \quad \forall s. P(s) \Rightarrow Q(\llbracket c \rrbracket(s)),$$

$$\mathbf{phaore} [c : P \Rightarrow Q] = 1 \quad \equiv \quad \Pr[\llbracket c \rrbracket(s) \text{ terminates and satisfies } Q \mid P(s)] = 1,$$

and

$$\mathbf{equiv} [c_1 \sim c_2 : P \Rightarrow Q] \quad \equiv \quad P(s_1, s_2) \Rightarrow Q(\llbracket c_1 \rrbracket(s_1), \llbracket c_2 \rrbracket(s_2)).$$

Exact Declaration Correspondence

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
Fastexp.ec:22	op	op fastexp_nbits N x n =	Exponentiation function; the theorem family proves $\text{fastexp}(x, n) = x^n$.
Fastexp.ec:26	op	op fastexp x n =	Exponentiation function; the theorem family proves $\text{fastexp}(x, n) = x^n$.
Fastexp.ec:57	lemma	lemma fastexp_nbitsP N x n :	Correctness of binary exponentiation: $\text{fastexp}(x, n) = x^n$.
Fastexp.ec:63	lemma	lemma fastexpP x n :	Correctness of binary exponentiation: $\text{fastexp}(x, n) = x^n$.
Fq.ec:18	op	op q : int = 64513.	Prime modulus $q = 64513$.
Fq.ec:19	lemma	lemma qE : q = 64513 by rewrite /q.	Field-arithmetic constant or word-representation theorem.
Fq.ec:22	op	op k <- 32,	Montgomery parameters: $k = 32$, $R = 2^{32}$, $q^{-1} \bmod R$, and $R^{-1} \bmod q$.
Fq.ec:23	op	op q <- q,	Prime modulus $q = 64513$.
Fq.ec:24	op	op qinv <- 940508161,	Montgomery parameters: $k = 32$, $R = 2^{32}$, $q^{-1} \bmod R$, and $R^{-1} \bmod q$.
Fq.ec:25	op	op Rinv <- 50386	Montgomery parameters: $k = 32$, $R = 2^{32}$, $q^{-1} \bmod R$, and $R^{-1} \bmod q$.
Fq.ec:38	lemma	lemma smod_W32 a :	Compatibility of centered modular representatives with Jasmin word signedness.
Fq.ec:42	lemma	lemma smod_W64 a :	Compatibility of centered modular representatives with Jasmin word signedness.
Fq.ec:52	lemma	lemma aux32_0 x :	Bit-level arithmetic-shift fact used to identify machine shifts with signed integer division.
Fq.ec:93	lemma	lemma aux32_1 x :	Bit-level arithmetic-shift fact used to identify machine shifts with signed integer division.
Fq.ec:134	lemma	lemma aux32_2 (a : W64.t) :	Bit-level arithmetic-shift fact used to identify machine shifts with signed integer division.
Fq.ec:155	lemma	lemma SAR_sem32 (a : W64.t) :	Bit-level arithmetic-shift fact used to identify machine shifts with signed integer division.
Fq.ec:193	lemma	lemma montgomery_reduce_corr_h_a :	Hoare/phoare theorem: Jasmin Montgomery reduction returns $\text{SREDC}(a)$.
Fq.ec:300	lemma	lemma montgomery_reduce_ll :	Losslessness/termination of the corresponding deterministic Jasmin procedure.
Fq.ec:303	lemma	lemma montgomery_reduce_corr_a :	Hoare/phoare theorem: Jasmin Montgomery reduction returns $\text{SREDC}(a)$.
Fq.ec:317	lemma	lemma fgmul_corr_h_a_b :	Hoare/phoare theorem: Jasmin multiplication returns $\text{SREDC}(ab)$.
Fq.ec:471	lemma	lemma fgmul_ll : islossless M._fgmul by proc; islossless.	Losslessness/termination of the corresponding deterministic Jasmin procedure.
Fq.ec:473	lemma	lemma fgmul_corr_a_b :	Hoare/phoare theorem: Jasmin multiplication returns $\text{SREDC}(ab)$.
Fq.ec:484	op	op bw32 (a : W32.t) (sz : int) =	Signed-word bound predicate: $\text{bw32}(w, b) \iff -2^b \leq \text{to_sint}(w) < 2^b$.
Fq.ec:487	lemma	lemma exp_max a b :	Signed-word range theorem for addition/subtraction: bounds grow by at most one exponent.
Fq.ec:494	lemma	lemma add_corr (a b : W32.t) (asz bsz : int) :	Signed-word range theorem for addition/subtraction: bounds grow by at most one exponent.
Fq.ec:515	lemma	lemma sub_corr (a b : W32.t) (asz bsz : int) :	Signed-word range theorem for addition/subtraction: bounds grow by at most one exponent.
GFq.ec:5	op	op q : int = 64513 axiomatized by qE.	Field modulus $q = 64513$.
GFq.ec:9	op	op p <- q	Total EasyCrypt function or predicate used by the NTT functional-correctness proof.
GFq.ec:16	op	op as_sint(x : coeff) = if (q-1) % 2 < asint x then asint x - q else asint x.	Centered representative $\text{sint}(x) = \text{asint}(x)$ if $\text{asint}(x) \leq (q-1)/2$, otherwise $\text{asint}(x) - q$.
GFq.ec:21	op	op round(x : real) : int = floor (x + inv 2%r).	Nearest-integer map $\lfloor x + 1/2 \rfloor$.
GFq.ec:24	op	op compress(d : int, x : coeff) : int = comp d (asint x)%r % 2^d.	$\text{compress}_d(x) = \lfloor \text{asint}(x) 2^d / q + 1/2 \rfloor \bmod 2^d$.
GFq.ec:27	op	op decompress(d : int, x : int) : coeff = incoeff (decomp d x)%r).	$\text{compress}_q(x) = \lfloor \text{asint}(x) 2^d / q + 1/2 \rfloor \bmod 2^d$.
Hpoly_extract.ec:186	module	module M = {	Extracted Jasmin transition system; its procedures act on concrete machine states and byte arrays.
Montgomery.ec:6	lemma	lemma div_pow x n m : 0 <= m <= n => 0 < x => x^n % x^m = x^(n - m).	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:14	lemma	lemma pow_div1 b k :	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:18	lemma	lemma pow_div2 b k :	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:22	lemma	lemma div_mulr p q m : m % q => (p * q) % m = p * (q % m).	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:28	lemma	lemma div_mull p q m : m % p => (p * q) % m = (p % m) * q.	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:33	lemma	lemma dvdNz (m n : int) : ((-m) % n) = (m % n).	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:36	lemma	lemma div_mul p m n : 0 <= m => m % p => p % (m * n) = p % m % n.	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:42	lemma	lemma divM_mul p q m n :	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:55	op	op k : { int 2 < k } as gt2_k.	Word exponent k ; the Montgomery radix is $R = 2^k$.
Montgomery.ec:57	op	op R = 2^k.	Word exponent k ; the Montgomery radix is $R = 2^k$.
Montgomery.ec:59	lemma	lemma dvd4R : 4 % R	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:64	lemma	lemma dvd2R : 2 % R	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:69	lemma	lemma gt0 R : 0 < R by apply: expr_gt0.	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:72	op	op q : int.	Total EasyCrypt function or predicate used by the NTT functional-correctness proof.
Montgomery.ec:77	op	op qinv : int.	Integer inverse parameter satisfying $qq^{-1} \equiv 1 \pmod{R}$.
Montgomery.ec:81	op	op Rinv : int.	Integer inverse parameter satisfying $RR^{-1} \equiv 1 \pmod{q}$ or modulo N .
Montgomery.ec:85	op	op smod (a b : int) =	Centered representative congruent to $a \bmod b$ in $[-b/2, b/2]$.
Montgomery.ec:90	lemma	lemma smod_bnd a b :	Centered modular-representative theorem: bounds, congruence, or compatibility with addition.
Montgomery.ec:94	lemma	lemma smod_congr a b : (smod a b) % b = a % b.	Centered modular-representative theorem: bounds, congruence, or compatibility with addition.
Montgomery.ec:101	lemma	lemma smod_sq t : smod (t % R) % (R ^ 2) R = smod (t % R) R.	Centered modular-representative theorem: bounds, congruence, or compatibility with addition.
Montgomery.ec:109	lemma	lemma wd_bnd x : 0 <= x % (R*R) % R < 'l R	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:112	lemma	lemma smod_small x y :	Centered modular-representative theorem: bounds, congruence, or compatibility with addition.
Montgomery.ec:130	lemma	lemma smod_div x : smod (x * R) (R ^ 2) % R = smod x R.	Centered modular-representative theorem: bounds, congruence, or compatibility with addition.
Montgomery.ec:143	lemma	lemma smod_exists x y : exists k, smod x y = x + k * y.	Centered modular-representative theorem: bounds, congruence, or compatibility with addition.
Montgomery.ec:150	lemma	lemma inrange a : 0 <= a < R % 2 => a % R = a by smt().	Centered modular-representative theorem: bounds, congruence, or compatibility with addition.
Montgomery.ec:151	lemma	lemma outrange a : - R%/2 <= a < 0 => a % R = R + a by smt(@IntDiv).	Centered modular-representative theorem: bounds, congruence, or compatibility with addition.
Montgomery.ec:153	lemma	lemma sign_comp a b : smod (a % R + b % R) R = smod (a + b) R	Centered modular-representative theorem: bounds, congruence, or compatibility with addition.
Montgomery.ec:158	op	op SREDC (a : int) : int =	Signed Montgomery reduction: output is congruent to $aR^{-1} \pmod{q}$ and lies in a signed interval.
Montgomery.ec:163	lemma	lemma SREDCp_corr (a : int) :	Montgomery correctness: output is bounded and congruent to multiplication by R^{-1} modulo the target modulus.
Montgomery.ec:276	op	op k : int.	Word exponent k ; the Montgomery radix is $R = 2^k$.
Montgomery.ec:279	op	op R = 2^k.	Word exponent k ; the Montgomery radix is $R = 2^k$.
Montgomery.ec:281	lemma	lemma R_gt0 : 0 < R by smt(expr_gt0).	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:284	op	op _N : int.	Odd modulus N for generic REDC.
Montgomery.ec:287	op	op _N' : int.	Montgomery inverse parameter N' for REDC.
Montgomery.ec:289	op	op Rinv : int.	Integer inverse parameter satisfying $RR^{-1} \equiv 1 \pmod{q}$ or modulo N .
Montgomery.ec:293	op	op REDC' (T : int) : int =	Classical Montgomery reduction $T \mapsto TR^{-1} \pmod{N}$ with conditional subtraction.
Montgomery.ec:297	lemma	lemma aux_divR T :	Integer divisibility/power lemma justifying Montgomery arithmetic and radix manipulation.
Montgomery.ec:308	lemma	lemma REDC'_congr T :	Montgomery correctness: output is bounded and congruent to multiplication by R^{-1} modulo the target modulus.
Montgomery.ec:322	lemma	lemma REDC'_bnds T n :	Montgomery correctness: output is bounded and congruent to multiplication by R^{-1} modulo the target modulus.
Montgomery.ec:349	op	op REDC T = let t = REDC' T in if _N <= t then t-_N else t.	Odd modulus N for generic REDC.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
Montgomery.ec:351	lemma	lemma REDC_corr T:	Montgomery correctness: output is bounded and congruent to multiplication by R^{-1} modulo the target modulus.
Montgomery.ec:368	op	op r: int.	Total EasyCrypt function or predicate used by the NTT functional-correctness proof.
Montgomery.ec:372	op	op w: int.	Total EasyCrypt function or predicate used by the NTT functional-correctness proof.
Montgomery.ec:376	op	op _N: int.	Odd modulus N for generic REDC.
Montgomery.ec:379	op	op k <- w,	Word exponent k ; the Montgomery radix is $R = 2^k$.
Montgomery.ec:380	op	op _N <- _N.	Odd modulus N for generic REDC.
Montgomery.ec:383	op	op REDCk : int -> int -> int.	Iterated Montgomery reduction with r limbs of width w .
Montgomery.ec:389	lemma	lemma REDCk_corr r T:	Montgomery correctness: output is bounded and congruent to multiplication by R^{-1} modulo the target modulus.
Montgomery.ec:415	op	op REDC T = let t = REDCk r T in if _N <= t then t-_N else t.	Odd modulus N for generic REDC.
Montgomery.ec:417	lemma	lemma REDC_corr T:	Montgomery correctness: output is bounded and congruent to multiplication by R^{-1} modulo the target modulus.
NTTEndToEnd.ec:12	type	type poly = coeff Array256.t.	$\text{poly} = \mathbb{F}_q^{256}$; when multiplication is present, the same carrier is read as $\mathbb{F}_q[X]/(X^{256} + 1)$.
NTTEndToEnd.ec:14	op	op jasmin_ntt_full_post (rp : BArray1024.t) (p : poly) =	Final forward postcondition: decoded output is $F(p)$ with the output bound.
NTTEndToEnd.ec:17	op	op jasmin_invntt_full_post (rp : BArray1024.t) (p : poly) =	Final inverse postcondition: decoded output is $R \cdot F^{-1}(p)$ with the output bound.
NTTEndToEnd.ec:21	lemma	lemma full_ntt_post_from_core rp r p :	Composition theorem: core Jasmin refinement plus algebraic loop theorem implies the final postcondition.
NTTEndToEnd.ec:27	lemma	lemma full_invntt_post_from_core rp r p :	Composition theorem: core Jasmin refinement plus algebraic loop theorem implies the final postcondition.
NTTEndToEnd.ec:33	lemma	lemma jasmin_ntt_full_from_core_and_alg p :	Composition theorem: core Jasmin refinement plus algebraic loop theorem implies the final postcondition.
NTTEndToEnd.ec:46	lemma	lemma jasmin_ntt_full p :	End-to-end Hoare theorem for a Jasmin NTT entry point.
NTTEndToEnd.ec:53	lemma	lemma jasmin_poly_ntt_jazz_full p :	End-to-end Hoare theorem for a Jasmin NTT entry point.
NTTEndToEnd.ec:60	lemma	lemma jasmin_invntt_full_from_core_and_alg p :	Composition theorem: core Jasmin refinement plus algebraic loop theorem implies the final postcondition.
NTTEndToEnd.ec:73	lemma	lemma jasmin_invntt_full p :	End-to-end Hoare theorem for a Jasmin NTT entry point.
NTTEndToEnd.ec:80	lemma	lemma jasmin_poly_invntt_jazz_full p :	End-to-end Hoare theorem for a Jasmin NTT entry point.
NTTEndToEnd.ec:87	lemma	lemma haetae_jasmin_ntt_correct p :	Final forward functional-correctness theorem: exported Jasmin computes F .
NTTEndToEnd.ec:95	lemma	lemma haetae_jasmin_invntt_correct p :	Final inverse functional-correctness theorem: exported Jasmin computes $R \cdot F^{-1}$.
NTTFullAlgebra.ec:11	type	type poly = coeff Array256.t.	$\text{poly} = \mathbb{F}_q^{256}$; when multiplication is present, the same carrier is read as $\mathbb{F}_q[X]/(X^{256} + 1)$.
NTTFullAlgebra.ec:13	lemma	lemma exp_zroot_512 :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:22	lemma	lemma inccoeff_m1 :	Arithmetic side condition used in the stage-invariant proof.
NTTFullAlgebra.ec:28	lemma	lemma exp_zroot_512_mul n :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:34	lemma	lemma exp_zroot_dvd512 e :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:43	lemma	lemma exp_zroot_mod512 e n :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:50	lemma	lemma exp_zroot_m256 :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:57	lemma	lemma pow2_div k :	Arithmetic side condition used in the stage-invariant proof.
NTTFullAlgebra.ec:68	lemma	lemma div256_pow2 k :	Arithmetic side condition used in the stage-invariant proof.
NTTFullAlgebra.ec:81	lemma	lemma pow25 k :	Arithmetic side condition used in the stage-invariant proof.
NTTFullAlgebra.ec:86	lemma	lemma div256_pair_range k bsj :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:104	lemma	lemma bsrev_add_pow2 k i :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:117	lemma	lemma bsrev_stage_index k ks :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:149	lemma	lemma zetas_stage k ks :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:165	lemma	lemma bsrev_extend_zero m x :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:179	lemma	lemma bsrev_extend_one m x :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:204	lemma	lemma bsrev_double m x :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:223	lemma	lemma bsrev_double_plus1 m x :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:242	lemma	lemma bsrev_ones n :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:260	lemma	lemma bsrev_zbase_index k ks :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:313	lemma	lemma bsrev_stage_pair_low k ks off :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:371	lemma	lemma bsrev_stage_pair_high k ks off :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:439	lemma	lemma bsrev8_low_dvd k i :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:452	lemma	lemma exp_zroot_stage_extra k i :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:472	lemma	lemma exp_zroot_low_shift k start i :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:490	lemma	lemma exp_zroot_high_shift k start i :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.
NTTFullAlgebra.ec:524	lemma	lemma partial_ntt_split_low p k start bsj :	Partial-sum recurrence: a length- 2^{k+1} sum decomposes into two length- 2^k sums with the appropriate twiddle.
NTTFullAlgebra.ec:568	lemma	lemma partial_ntt_split_high p k start bsj :	Partial-sum recurrence: a length- 2^{k+1} sum decomposes into two length- 2^k sums with the appropriate twiddle.
NTTFullAlgebra.ec:609	op	op partial_ntt_spec (r p : poly, len start bsj : int) =	Predicate equating one cell with $P_{\ell,s,b}(p)$.
NTTFullAlgebra.ec:613	op	op full_stage_spec (r p : poly, len : int) =	Forward stage invariant: every cell stores its current partial NTT sum.
NTTFullAlgebra.ec:619	op	op fwd_old_pair_spec (r p : poly, k ks off : int) =	Before-butterfly condition: two cells contain old length- 2^k partial sums.
NTTFullAlgebra.ec:623	op	op fwd_new_pair_spec (r p : poly, k ks off : int) =	After-butterfly condition: two cells contain new length- 2^{k+1} partial sums.
NTTFullAlgebra.ec:627	op	op fwd_pair_done (ks off ks' off' : int) =	Lexicographic progress predicate for visited butterfly pairs.
NTTFullAlgebra.ec:630	op	op fwd_stage_progress (r p : poly, k ks off : int) =	Mixed invariant: visited pairs satisfy new specs, unvisited pairs old specs.
NTTFullAlgebra.ec:639	op	op fwd_low_addr (k ks off : int) =	Lower forward butterfly address $ks2^{8-k} + \text{off}$.
NTTFullAlgebra.ec:642	op	op fwd_high_addr (k ks off : int) =	Upper forward butterfly address $ks2^{8-k} + \text{off} + 2^{7-k}$.
NTTFullAlgebra.ec:645	op	op set2_add_mulr (p : poly, z : coeff, a b : int) =	Forward butterfly update $(a, b) \mapsto (a + zb, a - zb)$.
NTTFullAlgebra.ec:648	lemma	lemma fwd_high_addrE k ks off :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:653	lemma	lemma fwd_old_low_addr k ks off :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:670	lemma	lemma fwd_old_high_addr k ks off :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:687	lemma	lemma fwd_new_low_addr k ks off :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:698	lemma	lemma fwd_new_high_addr k ks off :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:709	lemma	lemma block_low_div l x y :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:720	lemma	lemma block_low_mod l x y :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:730	lemma	lemma block_high_div l x y :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:741	lemma	lemma block_high_mod l x y :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:751	lemma	lemma fwd_pair_addr_disjoint k ks off ks' off' :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:842	lemma	lemma forward_pair_inputs k start bsj :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:866	lemma	lemma set2_add_mulr_eq1 p z a b :	Array-update theorem for the mathematical butterfly update.
NTTFullAlgebra.ec:875	lemma	lemma set2_add_mulr_eq2 p z a b :	Array-update theorem for the mathematical butterfly update.
NTTFullAlgebra.ec:886	lemma	lemma set2_add_mulr_neq p z a b x :	Array-update theorem for the mathematical butterfly update.
NTTFullAlgebra.ec:895	lemma	lemma set2_add_mulr_assign (p : poly) (z : coeff) (a b : int) :	Array-update theorem for the mathematical butterfly update.
NTTFullAlgebra.ec:909	lemma	lemma fwd_addr_ranges k ks off :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
NTTFullAlgebra.ec:935	lemma	lemma partial_ntt_spec_neq_update p r z len start bsj a b :	Arithmetic side condition used in the stage-invariant proof.
NTTFullAlgebra.ec:946	lemma	lemma fwd_old_pair_spec_neq_update p r z k ks off ks' off' :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:985	lemma	lemma fwd_new_pair_spec_neq_update p r z k ks off ks' off' :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:1024	lemma	lemma fwd_stage_progress_init p r k :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1059	lemma	lemma fwd_stage_progress_old p r k ks off ks' off' :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1071	lemma	lemma fwd_stage_progress_new p r k ks off ks' off' :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1083	lemma	lemma forward_pair_refines_values p r k start bsj :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:1145	lemma	lemma forward_pair_refines p r k start bsj :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:1163	lemma	lemma forward_nat_pair_refines p r k ks off :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:1203	lemma	lemma fwd_stage_progress_step p r k ks off :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1259	lemma	lemma fwd_stage_progress_next_block p r k ks :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1277	lemma	lemma div256_pow2_next k :	Arithmetic side condition used in the stage-invariant proof.
NTTFullAlgebra.ec:1291	lemma	lemma fwd_stage_progress_finish p r k :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1376	lemma	lemma pow2_stage_product k :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1388	lemma	lemma pow2_stage_block k :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1398	lemma	lemma fwd_len_div2_next k :	Arithmetic side condition used in the stage-invariant proof.
NTTFullAlgebra.ec:1412	op	op fwd_outer_inv (p r zetas : poly, len zetasctr : int) =	Hoare loop invariant connecting loop counters to the forward stage-progress predicate.
NTTFullAlgebra.ec:1423	op	op fwd_middle_inv (p r zetas : poly, len start zetasctr : int) =	Hoare loop invariant connecting loop counters to the forward stage-progress predicate.
NTTFullAlgebra.ec:1433	op	op fwd_inner_inv	Hoare loop invariant connecting loop counters to the forward stage-progress predicate.
NTTFullAlgebra.ec:1447	lemma	lemma fwd_outer_inv_init p :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1466	lemma	lemma fwd_outer_exit_stage p r zetas len zetasctr :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1482	lemma	lemma fwd_outer_enter_middle p r zetas len zetasctr :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1503	lemma	lemma fwd_middle_enter_inner p r zetas len start zetasctr :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1529	lemma	lemma fwd_inner_step_set2 p r zetas len start j zetasctr zeta_ :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1570	lemma	lemma fwd_inner_step_array p r zetas len start j zetasctr zeta_ :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1624	lemma	lemma fwd_inner_exit_middle p r zetas len start j zetasctr zeta_ :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1656	lemma	lemma fwd_middle_exit_outer p r zetas len start zetasctr :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1701	lemma	lemma full_stage1_init (p : poly) :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1714	lemma	lemma full_stage256_imp p r :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1726	lemma	lemma fwd_outer_exit_full p r zetas len zetasctr :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1735	lemma	lemma ntt_full_ntt (p : poly) :	Hoare theorem: imperative forward loop computes F .
NTTFullAlgebra.ec:1760	op	op inv_low_addr (k ks off : int) =	Lower inverse butterfly address $ks2^{k+1} + off$.
NTTFullAlgebra.ec:1763	op	op inv_high_addr (k ks off : int) =	Upper inverse butterfly address $ks2^{k+1} + off + 2^k$.
NTTFullAlgebra.ec:1766	lemma	lemma inv_high_addrE k ks off :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:1771	lemma	lemma inv_stage_blocks_product k :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1783	lemma	lemma inv_pair_inputs k ks off :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:1802	lemma	lemma bsrev_inv_pair_low k ks off :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:1837	lemma	lemma bsrev_inv_pair_high k ks off :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:1876	lemma	lemma zetas_inv_stage k ks :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:1917	lemma	lemma inv_low_twiddle_factor k ks off :	Arithmetic side condition used in the stage-invariant proof.
NTTFullAlgebra.ec:1945	lemma	lemma inv_high_twiddle_factor k ks off :	Arithmetic side condition used in the stage-invariant proof.
NTTFullAlgebra.ec:1986	op	op partial_invntt (p : poly, len start bsj : int) =	$Q_{\ell,s,b}(p) = \sum_{u=0}^{\ell-1} p b \ell + u \zeta^{- (2 \text{brevg}(b \ell + u) + 1) s}$.
NTTFullAlgebra.ec:1993	op	op partial_invntt_spec (r p : poly, len start bsj : int) =	Predicate equating one cell with the inverse partial sum $Q_{\ell,s,b}(p)$.
NTTFullAlgebra.ec:1996	op	op inv_stage_spec (r p : poly, len : int) =	Inverse stage invariant: every cell stores its current inverse partial sum.
NTTFullAlgebra.ec:2002	lemma	lemma partial_invntt_split_low p k off ks :	Partial-sum recurrence: a length- 2^{k+1} sum decomposes into two length- 2^k sums with the appropriate twiddle.
NTTFullAlgebra.ec:2034	lemma	lemma partial_invntt_split_high p k off ks :	Partial-sum recurrence: a length- 2^{k+1} sum decomposes into two length- 2^k sums with the appropriate twiddle.
NTTFullAlgebra.ec:2135	op	op inv_old_pair_spec (r p : poly, k ks off : int) =	Before inverse butterfly: two cells contain old inverse partial sums.
NTTFullAlgebra.ec:2139	op	op inv_new_pair_spec (r p : poly, k ks off : int) =	After inverse butterfly: two cells contain merged inverse partial sums.
NTTFullAlgebra.ec:2143	op	op inv_pair_done (ks off ks' off' : int) =	Lexicographic progress predicate for visited butterfly pairs.
NTTFullAlgebra.ec:2146	op	op inv_stage_progress (r p : poly, k ks off : int) =	Mixed invariant for visited/unvisited inverse butterfly pairs.
NTTFullAlgebra.ec:2155	op	op set2_inv_mulr (p : poly, z : coeff, a b : int) =	Inverse butterfly update $(a, b) \mapsto (a + b, z(a - b))$.
NTTFullAlgebra.ec:2158	lemma	lemma inv_pair_addr_disjoint k ks off ks' off' :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:2244	lemma	lemma set2_inv_mulr_eq1 p z a b :	Array-update theorem for the mathematical butterfly update.
NTTFullAlgebra.ec:2255	lemma	lemma set2_inv_mulr_eq2 p z a b :	Array-update theorem for the mathematical butterfly update.
NTTFullAlgebra.ec:2263	lemma	lemma set2_inv_mulr_neq p z a b x :	Array-update theorem for the mathematical butterfly update.
NTTFullAlgebra.ec:2272	lemma	lemma set2_inv_mulr_assign (p : poly) (z : coeff) (a b : int) :	Array-update theorem for the mathematical butterfly update.
NTTFullAlgebra.ec:2278	lemma	lemma partial_invntt_spec_neq_update p r z len start bsj a b :	Arithmetic side condition used in the stage-invariant proof.
NTTFullAlgebra.ec:2289	lemma	lemma inv_old_pair_spec_neq_update p r z k ks off ks' off' :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:2334	lemma	lemma inv_new_pair_spec_neq_update p r z k ks off ks' off' :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:2379	lemma	lemma inv_stage_progress_init p r k :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2404	lemma	lemma inv_stage_progress_old p r k ks off ks' off' :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2416	lemma	lemma inv_stage_progress_new p r k ks off ks' off' :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2428	lemma	lemma inverse_pair_refines_values p r k ks off :	Addressing and pair-refinement theorem identifying the two cells touched by a butterfly.
NTTFullAlgebra.ec:2472	lemma	lemma inv_stage_progress_step p r k ks off :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2528	lemma	lemma inv_stage_progress_next_block p r k ks :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2546	lemma	lemma inv_stage_progress_finish p r k :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2589	op	op inv_outer_inv (p r zetas_inv : poly, len zetasctr : int) =	Hoare loop invariant for inverse stages and final [256] ⁻¹ scaling.
NTTFullAlgebra.ec:2600	op	op inv_middle_inv (p r zetas_inv : poly, len start zetasctr : int) =	Hoare loop invariant for inverse stages and final [256] ⁻¹ scaling.
NTTFullAlgebra.ec:2610	op	op inv_inner_inv	Hoare loop invariant for inverse stages and final [256] ⁻¹ scaling.
NTTFullAlgebra.ec:2624	op	op inv_post_inv (p r zetas_inv : poly, j : int) =	Hoare loop invariant for inverse stages and final [256] ⁻¹ scaling.
NTTFullAlgebra.ec:2633	lemma	lemma inv_stage1_init (p : poly) :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2645	lemma	lemma inv_outer_inv_init p :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2657	lemma	lemma inv_outer_enter_middle p r zetas_inv len zetasctr :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2678	lemma	lemma inv_middle_enter_inner p r zetas_inv len start zetasctr :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2707	lemma	lemma inv_inner_step_set2 p r zetas_inv len start j zetasctr zeta_ :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2748	lemma	lemma inv_inner_step_array p r zetas_inv len start j zetasctr zeta_ :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2830	lemma	lemma inv_inner_step_array_wp p r zetas_inv len start j zetasctr zeta_ :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2852	lemma	lemma inv_inner_exit_middle p r zetas_inv len start j zetasctr zeta_ :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2885	lemma	lemma inv_middle_exit_outer p r zetas_inv len start zetasctr :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2934	lemma	lemma inv_outer_exit_stage p r zetas_inv len zetasctr :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2955	lemma	lemma inv_outer_exit_post p r zetas_inv len zetasctr :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:2977	lemma	lemma inv_post_step p r zetas_inv j :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:3008	lemma	lemma zetas_inv_255 :	Root-of-unity and bit-reversal algebra aligning loop twiddle indices with spectral exponents.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
NTTFullAlgebra.ec:3014	lemma	lemma inv_post_exit_full p r zetas_inv j :	Loop-invariant theorem: initialization, preservation, or exit of a forward/inverse NTT stage predicate.
NTTFullAlgebra.ec:3055	lemma	lemma invntt_full_invntt (p : poly) :	Hoare theorem: imperative inverse loop computes F^{-1} .
NTTFullSpec.ec:10	lemma	lemma pow2_8 :	Concrete equality used to distinguish and evaluate NTT specifications.
NTTFullSpec.ec:14	lemma	lemma bsrev8_range_256 n :	Bit-reversal range or involution theorem over $[0, 256)$.
NTTFullSpec.ec:21	lemma	lemma bsrev8_involution_256 n :	Bit-reversal range or involution theorem over $[0, 256)$.
NTTFullSpec.ec:32	op	op partial_ntt (p : poly, len start bsj : int) =	$P_{\ell,s,b}(p) = \sum_{u=0}^{\ell-1} \zeta^{(2s+1)\text{brevs}(u)} p_{\text{brevs}(b\ell+u)}$.
NTTFullSpec.ec:39	op	op full_ntt_spec (r p : poly) =	Extensional specification: $r_{\text{brevs}(s)} = P_{256,s,0}(p)$ for all s .
NTTFullSpec.ec:44	op	op full_ntt (p : poly) : poly =	$F(p)_i = \sum_{j=0}^{255} p_j \zeta^{(2\text{brevs}(i)+1)j}$.
NTTFullSpec.ec:50	op	op full_invntt (p : poly) : poly =	$F^{-1}(p)_i = [256]^{-1} \sum_{j=0}^{255} p_j \zeta^{-(2\text{brevs}(j)+1)i}$.
NTTFullSpec.ec:58	op	op full_invntt_spec (r p : poly) =	Extensional inverse specification in bit-reversed output order.
NTTFullSpec.ec:68	lemma	lemma full_ntt_spec_imp p r :	Extensional equality: the indexed forward specification equals the array function F .
NTTFullSpec.ec:101	lemma	lemma full_invntt_spec_imp p r :	Extensional equality: the indexed inverse specification equals F^{-1} .
NTTFullSpec.ec:115	lemma	lemma one_get0 :	Concrete equality used to distinguish and evaluate NTT specifications.
NTTFullSpec.ec:119	lemma	lemma one_get_neq0 i :	Concrete equality used to distinguish and evaluate NTT specifications.
NTTFullSpec.ec:129	lemma	lemma rq_ntt_one_odd1 :	Concrete equality used to distinguish and evaluate NTT specifications.
NTTFullSpec.ec:138	lemma	lemma full_ntt_one_odd1 :	Concrete equality used to distinguish and evaluate NTT specifications.
NTTFullSpec.ec:151	lemma	lemma rq_ntt_is_not_full_ntt_on_one :	Separates the Kyber-style split transform from the HAETAE full NTT target by a counterexample.
NTT_Fq.ec:12	type	type CR.t <- coeff,	Carrier of the cloned commutative-ring/field theory, instantiated as $\mathbb{F}_q$.
NTT_Fq.ec:13	op	op CR.zeror <- Zq.zero,	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
NTT_Fq.ec:14	op	op CR.oner <- Zq.one,	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
NTT_Fq.ec:15	op	op CR.(+) <- Zq.(+),	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
NTT_Fq.ec:16	op	op CR.([-]) <- Zq.([-]),	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
NTT_Fq.ec:17	op	op CR.(*) <- Zq.(*) ,	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
NTT_Fq.ec:18	op	op CR.invr <- Zq.inv,	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
NTT_Fq.ec:19	op	op CR.exp <- Zq.exp,	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
NTT_Fq.ec:20	op	op CR.ofint <- ZqField.ofint,	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
NTT_Fq.ec:47	module	module NTT = {	Imperative transition system implementing the forward and inverse NTT loops over $\mathbb{F}_q^{256}$.
NTT_Fq.ec:113	lemma	lemma ntt_middle_right start len j0 :	Loop-index arithmetic or representation theorem for the imperative NTT model.
NTT_Fq.ec:135	lemma	lemma inv_middle_right start len j0 :	Loop-index arithmetic or representation theorem for the imperative NTT model.
NTT_Fq.ec:157	lemma	lemma inv_outer_progress len :	Loop-index arithmetic or representation theorem for the imperative NTT model.
NTT_Fq.ec:181	lemma	lemma ntt_init_progress len0 :	Loop-index arithmetic or representation theorem for the imperative NTT model.
NTT_Fq.ec:197	lemma	lemma ntt_spec_ll : islossless NTT.ntt.	Losslessness/termination of the imperative EasyCrypt NTT loop.
NTT_Fq.ec:259	lemma	lemma invntt_spec_ll : islossless NTT.invntt.	Losslessness/termination of the imperative EasyCrypt NTT loop.
NTT_Fq.ec:322	op	op perm256 ['t] (p : int -> int) (a : 't Array256.t) :	Permutation action $a \mapsto (i \mapsto a(p(i)))$.
NTT_Fq.ec:325	op	op bsrev256 ['t] : 't Array256.t -> 't Array256.t =	Permutation by brevs on $\mathbb{F}_q^{256}$.
NTT_Fq.ec:328	op	op R = incoeff Fq.SignedReductions.R.	Field element $R = [2^{32}]_q$ used for Montgomery representation.
NTT_Fq.ec:331	op	op scale255 = incoeff (-252).	Field element $[256]^{-1} = [-252]_q$.
NTT_Fq.ec:333	op	op array256_mont_inv (p : coeff Array256.t) =	Pointwise multiplication by R , used to cancel/expose Montgomery representation.
NTT_Fq.ec:336	op	op word_to_coeff (w : W32.t) : coeff =	Word decoding $w \mapsto [\text{to_sint}(w)]_q$.
NTT_Fq.ec:339	op	op barray256_to_poly (rp : BArray1024.t) : coeff Array256.t =	Decoded polynomial $D(rp)_i = [\text{to_sint}(\text{get32}(rp, i))]_q$.
NTT_Fq.ec:342	op	op poly_repr (rp : BArray1024.t) (p : coeff Array256.t) : bool =	Representation predicate $D(rp) = p$.
NTT_Fq.ec:345	op	op barray256_bound (rp : BArray1024.t) (sz : int) : bool =	Uniform per-coordinate signed-word bound.
NTT_Fq.ec:348	op	op poly_repr_bound (rp : BArray1024.t) (p : coeff Array256.t) (sz :	$D(rp) = p$ and every word satisfies the signed bound.
NTT_Fq.ec:351	lemma	lemma bw32_weaken (w : W32.t) sz1 sz2 :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:363	lemma	lemma barray256_bound_weaken rp sz1 sz2 :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:372	lemma	lemma poly_repr_bound_repr rp p sz :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:377	lemma	lemma poly_repr_bound_bound rp p sz :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:382	lemma	lemma barray256_to_polyE rp i :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:390	lemma	lemma poly_repr_get rp p i :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:400	lemma	lemma poly_repr_bound_get rp p sz i :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:412	lemma	lemma barray256_to_poly_set32E rp i w j :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:439	lemma	lemma poly_repr_set32 rp p i w :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:451	lemma	lemma barray256_bound_set32 rp i w sz :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:470	lemma	lemma poly_repr_bound_set32 rp p i w sz :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:483	lemma	lemma word_to_coeff_of_int z :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:491	lemma	lemma w32_q_to_sint :	Loop-index arithmetic or representation theorem for the imperative NTT model.
NTT_Fq.ec:497	lemma	lemma w32_negq_to_sint :	Loop-index arithmetic or representation theorem for the imperative NTT model.
NTT_Fq.ec:503	lemma	lemma word_to_coeff_small z :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:518	lemma	lemma incoeff_signed_repr z :	Loop-index arithmetic or representation theorem for the imperative NTT model.
NTT_Fq.ec:527	lemma	lemma word_to_coeff_repr z :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:536	lemma	lemma word_to_coeff_neg16505 :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:542	lemma	lemma word_to_coeff_26964 :	Representation theorem connecting W32/byte-array updates with $\mathbb{F}_q^{256}$ values and signed bounds.
NTT_Fq.ec:555	lemma	lemma unit_zroot : Zq.unit zroot.	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:558	lemma	lemma inv_zroot : Zq.inv zroot = incoeff 6209.	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:565	op	op zetas_inv : coeff Array256.t =	Inverse twiddle table: powers of ζ in inverse loop order, with final entry $[256]^{-1}$ in Montgomery form.
NTT_Fq.ec:569	lemma	lemma zetas_invE: zetas_inv = Array256.of_list witness	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:622	lemma	lemma scale255E :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:631	lemma	lemma unit_R :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:637	lemma	lemma inv_R :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:649	lemma	lemma zetas_inv_vals : array256_mont_inv zetas_inv =	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:697	lemma	lemma jzetas_inv_255_get32E :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:706	lemma	lemma jzetas_inv_255_wordE :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:713	lemma	lemma jzetas_inv_poly_vals i :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:730	lemma	lemma jzetas_inv_get i :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
NTT_Fq.ec:739	lemma	lemma jzetas_inv_polyE :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:753	lemma	lemma zetas_inv_255_montE :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:765	lemma	lemma jzetas_inv_255_scaleE :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:772	op	op array256_mont (p : coeff Array256.t) =	Pointwise Montgomery scaling map on $\mathbb{F}_q^{256}$.
NTT_Fq.ec:781	op	op zetas : coeff Array256.t =	Forward twiddle table: powers of ζ in forward loop order.
NTT_Fq.ec:784	lemma	lemma zetasE:	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:835	lemma	lemma zetas_vals : array256_mont zetas =	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:876	lemma	lemma jzetas_1_poly :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:885	lemma	lemma jzetas_1_word_montE :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:892	lemma	lemma jzetas_1_get32E :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:900	lemma	lemma jzetas_1_get32dE :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:908	lemma	lemma jzetas_polyE :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:935	lemma	lemma jzetas_poly_vals i :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:946	lemma	lemma jzetas_get i :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:954	lemma	lemma jzetas_1_montE :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:960	lemma	lemma jzetas_1E :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:967	lemma	lemma jzetas_1_wordE :	Twiddle-table, root-of-unity, or Montgomery-scaling equality in $\mathbb{F}_q$.
NTT_Fq.ec:974	lemma	lemma exp_zroot_256 : Zq.exp zroot 256 = incoeff (-1).	Loop-index arithmetic or representation theorem for the imperative NTT model.
RefJasminNTT.ec:21	lemma	lemma int_shr1_div2 x :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:29	lemma	lemma int_shl1_mul2 x :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:35	lemma	lemma word_to_coeff_add (a b : W32.t) (asz bsz : int) :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:57	lemma	lemma word_to_coeff_sub (a b : W32.t) (asz bsz : int) :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:79	lemma	lemma forward_butterfly_bounds s t asz tsz :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:93	lemma	lemma inverse_butterfly_bounds t0 coeff asz bsz :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:107	lemma	lemma unit_R_ref : Zq.unit NTT_Fq.R.	Montgomery/twiddle-table cancellation connecting Jasmin constants with $\mathbb{F}_q$ constants.
RefJasminNTT.ec:112	lemma	lemma inv_R_ref :	Montgomery/twiddle-table cancellation connecting Jasmin constants with $\mathbb{F}_q$ constants.
RefJasminNTT.ec:121	lemma	lemma array256_mont_get p i :	Montgomery/twiddle-table cancellation connecting Jasmin constants with $\mathbb{F}_q$ constants.
RefJasminNTT.ec:129	lemma	lemma mont_cancel p i :	Montgomery/twiddle-table cancellation connecting Jasmin constants with $\mathbb{F}_q$ constants.
RefJasminNTT.ec:140	lemma	lemma jzetas_get_cancel i :	Montgomery/twiddle-table cancellation connecting Jasmin constants with $\mathbb{F}_q$ constants.
RefJasminNTT.ec:151	lemma	lemma array256_mont_inv_get p i :	Montgomery/twiddle-table cancellation connecting Jasmin constants with $\mathbb{F}_q$ constants.
RefJasminNTT.ec:159	lemma	lemma mont_inv_cancel p i :	Montgomery/twiddle-table cancellation connecting Jasmin constants with $\mathbb{F}_q$ constants.
RefJasminNTT.ec:170	lemma	lemma jzetas_inv_get_cancel i :	Montgomery/twiddle-table cancellation connecting Jasmin constants with $\mathbb{F}_q$ constants.
RefJasminNTT.ec:182	lemma	lemma jzetas_inv_255_scale_cancel :	Montgomery/twiddle-table cancellation connecting Jasmin constants with $\mathbb{F}_q$ constants.
RefJasminNTT.ec:189	lemma	lemma fqmul_bw32_16 aa bb :	Jasmin multiplication refinement: decoded result equals field multiplication/Montgomery product and preserves bounds.
RefJasminNTT.ec:209	lemma	lemma fqmul_word_to_coeff_mul aa bb :	Jasmin multiplication refinement: decoded result equals field multiplication/Montgomery product and preserves bounds.
RefJasminNTT.ec:230	lemma	lemma fqmul_word_to_coeff_mul_h aa bb :	Jasmin multiplication refinement: decoded result equals field multiplication/Montgomery product and preserves bounds.
RefJasminNTT.ec:252	lemma	lemma fqmul_word_to_coeff_mul_bound_h aa bb :	Jasmin multiplication refinement: decoded result equals field multiplication/Montgomery product and preserves bounds.
RefJasminNTT.ec:279	lemma	lemma fqmul_word_to_coeff_mul_bound_ph aa bb :	Jasmin multiplication refinement: decoded result equals field multiplication/Montgomery product and preserves bounds.
RefJasminNTT.ec:290	op	op barray256_bound_by (rp : BArray1024.t) (bd : int -> int) : bool =	Pointwise signed-word bound with index-dependent exponent $bd(i)$, combined with representation when needed.
RefJasminNTT.ec:293	op	op poly_repr_bound_by	Pointwise signed-word bound with index-dependent exponent $bd(i)$, combined with representation when needed.
RefJasminNTT.ec:297	lemma	lemma barray256_bound_by_get rp bd i :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:303	lemma	lemma barray256_bound_by_weaken rp bd1 bd2 :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:313	lemma	lemma barray256_bound_by_set32 rp bd i w :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:332	lemma	lemma barray256_bound_by_set32_change rp bd bd' i w :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:354	lemma	lemma poly_repr_bound_by_repr rp p bd :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:359	lemma	lemma poly_repr_bound_by_bound rp p bd :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:364	lemma	lemma fqmul_product_bound_16_24 a b :	Jasmin multiplication refinement: decoded result equals field multiplication/Montgomery product and preserves bounds.
RefJasminNTT.ec:375	lemma	lemma jzetas_bound16 i :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:388	lemma	lemma jzetas_inv_bound16 i :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:401	op	op const_bound (sz : int) : int -> int = fun _ => sz.	Constant bound function $i \mapsto sz$.
RefJasminNTT.ec:403	lemma	lemma barray256_bound_by_const rp sz :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:408	lemma	lemma barray256_bound_by_const_bound rp sz :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:413	op	op fwd_len_ok (len : int) : bool =	Permitted loop length in the Jasmin loop schedule.
RefJasminNTT.ec:417	op	op fwd_stage_bound (len : int) : int =	Index-dependent exponent bounding signed machine words during the corresponding loop phase.
RefJasminNTT.ec:427	op	op fwd_zbase (len : int) : int =	Affine base index into the Jasmin twiddle table.
RefJasminNTT.ec:437	op	op fwd_middle_bound (sz start i : int) : int =	Index-dependent exponent bounding signed machine words during the corresponding loop phase.
RefJasminNTT.ec:440	op	op fwd_inner_bound (sz len start j i : int) : int =	Index-dependent exponent bounding signed machine words during the corresponding loop phase.
RefJasminNTT.ec:445	lemma	lemma fwd_stage_bound_range len :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:450	lemma	lemma fwd_stage_bound_pos len :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:455	lemma	lemma fwd_stage_bound_lt31 len :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:460	lemma	lemma fwd_stage_bound_ge16 len :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:465	lemma	lemma fwd_stage_next len :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:471	lemma	lemma fwd_len_next_ok len :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:477	lemma	lemma fwd_zbase_next len :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:483	lemma	lemma fwd_zbase_range len :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:488	lemma	lemma fwd_read_range len start zc :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:496	lemma	lemma fwd_block_end_bound len start zc :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:504	lemma	lemma fwd_stage_zbase_exit len zc :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:511	lemma	lemma fwd_middle_to_inner sz len start i :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:515	lemma	lemma fwd_inner_current sz len start j :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:525	lemma	lemma fwd_inner_weaken_unchanged sz len start j k :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:536	lemma	lemma fwd_inner_exit_to_middle sz len start i :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:570	lemma	lemma fwd_middle_exit_const sz start i :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:576	lemma	lemma fwd_middle_start_const sz i :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:580	lemma	lemma forward_twiddle_product	One Jasmin butterfly preserves the decoded polynomial according to the forward or inverse partial-sum formula.
RefJasminNTT.ec:612	lemma	lemma forward_twiddle_product_bound_h	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:633	lemma	lemma forward_twiddle_product_bound_ph	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:653	lemma	lemma forward_twiddle_product_bound_call_h	Signed-word bound or loop-index side condition for extracted Jasmin code.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
RefJasminNTT.ec:686	lemma	lemma forward_twiddle_product_bound_call_ph	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:704	lemma	lemma forward_twiddle_value	One Jasmin butterfly preserves the decoded polynomial according to the forward or inverse partial-sum formula.
RefJasminNTT.ec:728	lemma	lemma forward_butterfly_store	One Jasmin butterfly preserves the decoded polynomial according to the forward or inverse partial-sum formula.
RefJasminNTT.ec:776	lemma	lemma forward_butterfly_store_bound	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:817	lemma	lemma forward_butterfly_store_bound_by	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:866	lemma	lemma forward_spec_updateE (p : coeff Array256.t) j len t :	One Jasmin butterfly preserves the decoded polynomial according to the forward or inverse partial-sum formula.
RefJasminNTT.ec:878	lemma	lemma forward_butterfly_step_bound_by	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:922	lemma	lemma forward_twiddle_product_bound_from_inner	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:956	lemma	lemma forward_butterfly_repr	One Jasmin butterfly preserves the decoded polynomial according to the forward or inverse partial-sum formula.
RefJasminNTT.ec:985	lemma	lemma forward_butterfly_step	One Jasmin butterfly preserves the decoded polynomial according to the forward or inverse partial-sum formula.
RefJasminNTT.ec:1014	op	op inv_len_ok (len : int) : bool =	Permitted loop length in the Jasmin loop schedule.
RefJasminNTT.ec:1019	op	op inv_stage_bound (len : int) : int =	Index-dependent exponent bounding signed machine words during the corresponding loop phase.
RefJasminNTT.ec:1029	op	op inv_zbase (len : int) : int =	Affine base index into the Jasmin twiddle table.
RefJasminNTT.ec:1039	op	op inv_middle_bound (sz start i : int) : int =	Index-dependent exponent bounding signed machine words during the corresponding loop phase.
RefJasminNTT.ec:1042	op	op inv_inner_bound (sz len start j i : int) : int =	Index-dependent exponent bounding signed machine words during the corresponding loop phase.
RefJasminNTT.ec:1047	lemma	lemma inv_stage_bound_range len :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1052	lemma	lemma inv_stage_bound_pos len :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1057	lemma	lemma inv_stage_bound_lt31 len :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1062	lemma	lemma inv_stage_bound_ge16 len :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1067	lemma	lemma inv_stage_next len :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:1073	lemma	lemma inv_len_next_ok len :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1079	lemma	lemma inv_zbase_range len :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1084	lemma	lemma inv_read_range len start zc :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1092	lemma	lemma inv_block_end_bound len start zc :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1100	lemma	lemma inv_stage_zbase_exit len zc :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1107	lemma	lemma inv_middle_to_inner sz len start i :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:1111	lemma	lemma inv_inner_current sz len start j :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1121	lemma	lemma inv_inner_weaken_unchanged sz len start j k :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:1132	lemma	lemma inv_inner_exit_to_middle sz len start i :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:1166	lemma	lemma inv_middle_exit_const sz start i :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:1172	lemma	lemma inv_middle_start_const sz i :	Jasmin-to-mathematics refinement lemma.
RefJasminNTT.ec:1176	lemma	lemma inverse_spec_updateE (p : coeff Array256.t) z j len :	One Jasmin butterfly preserves the decoded polynomial according to the forward or inverse partial-sum formula.
RefJasminNTT.ec:1191	lemma	lemma inverse_twiddle_product_bound_call_h	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1214	lemma	lemma inverse_twiddle_product_bound_call_ph	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1230	lemma	lemma inverse_twiddle_value	One Jasmin butterfly preserves the decoded polynomial according to the forward or inverse partial-sum formula.
RefJasminNTT.ec:1250	lemma	lemma inverse_butterfly_store	One Jasmin butterfly preserves the decoded polynomial according to the forward or inverse partial-sum formula.
RefJasminNTT.ec:1304	lemma	lemma inverse_butterfly_store_bound	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1359	lemma	lemma inverse_butterfly_store_bound_by	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1417	lemma	lemma inverse_butterfly_step_bound_by	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1463	lemma	lemma inverse_twiddle_product_bound_from_inner	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1498	lemma	lemma inverse_twiddle_product_call_bound_from_inner	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1536	lemma	lemma inverse_butterfly_step_full	One Jasmin butterfly preserves the decoded polynomial according to the forward or inverse partial-sum formula.
RefJasminNTT.ec:1577	lemma	lemma inverse_butterfly_repr	One Jasmin butterfly preserves the decoded polynomial according to the forward or inverse partial-sum formula.
RefJasminNTT.ec:1607	op	op final_mont_array (p : coeff Array256.t) (j : int) : coeff Array256.t =	Partially scaled output vector: first j entries multiplied by the final Montgomery factor.
RefJasminNTT.ec:1610	op	op final_bound (j i : int) : int =	Index-dependent exponent bounding signed machine words during the corresponding loop phase.
RefJasminNTT.ec:1613	lemma	lemma final_mont_array_start p :	Montgomery/twiddle-table cancellation connecting Jasmin constants with $\mathbb{F}_q$ constants.
RefJasminNTT.ec:1620	lemma	lemma final_mont_array_exit p :	Montgomery/twiddle-table cancellation connecting Jasmin constants with $\mathbb{F}_q$ constants.
RefJasminNTT.ec:1631	lemma	lemma final_mont_array_get_raw p j :	Montgomery/twiddle-table cancellation connecting Jasmin constants with $\mathbb{F}_q$ constants.
RefJasminNTT.ec:1639	lemma	lemma final_mont_array_setE p j :	Montgomery/twiddle-table cancellation connecting Jasmin constants with $\mathbb{F}_q$ constants.
RefJasminNTT.ec:1662	lemma	lemma final_bound_start i :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1666	lemma	lemma final_bound_next_unchanged j k :	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1673	lemma	lemma inverse_final_product_bound_call_h	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1698	lemma	lemma inverse_final_product_bound_call_ph	Signed-word bound or loop-index side condition for extracted Jasmin code.
RefJasminNTT.ec:1713	lemma	lemma inverse_final_product_bound_from_inv rp j :	Signed-word bound or loop-index side condition for extracted Jasmin code.
Rq.ec:17	type	type poly = coeff Array256.t.	$\text{poly} = \mathbb{F}_q^{256}$; when multiplication is present, the same carrier is read as $\mathbb{F}_q[X]/(X^{256} + 1)$.
Rq.ec:19	op	op zero : poly = Array256.create Zq.zero.	Zero polynomial: $(0, \dots, 0)$.
Rq.ec:20	op	op one : poly = zero.[<Zq.one].	Zero polynomial: $(0, \dots, 0)$.
Rq.ec:25	op	op (&*) (pa pb : poly) : poly =	Negacyclic product in $\mathbb{F}_q[X]/(X^{256} + 1)$: coefficients wrap with sign because $X^{256} = -1$.
Rq.ec:32	op	op (&+) (pa pb : poly) : poly =	Pointwise vector addition over $\mathbb{F}_q$.
Rq.ec:35	op	op (&-) (p : poly) : poly = map Zq.([-]) p.	Pointwise additive inverse over $\mathbb{F}_q$.
Rq.ec:39	op	op compress_poly(d : int, p : poly) : int Array256.t = map (compress d) P.	Pointwise coefficient compression on $\mathbb{F}_q^{256}$.
Rq.ec:41	op	op decompress_poly(d : int, p : int Array256.t) : poly = map (decompress d) p.	Pointwise coefficient compression on $\mathbb{F}_q^{256}$.
Rq.ec:61	type	type CR.t <- coeff,	Carrier of the cloned commutative-ring/field theory, instantiated as $\mathbb{F}_q$.
Rq.ec:62	op	op CR.zeror <- Zq.zero,	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
Rq.ec:63	op	op CR.oner <- Zq.one,	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
Rq.ec:64	op	op CR.(+) <- Zq.(+),	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
Rq.ec:65	op	op CR.([-]) <- Zq.([-]),	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
Rq.ec:66	op	op CR.(*) <- Zq.(*),	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
Rq.ec:67	op	op CR.invr <- Zq.invr,	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
Rq.ec:68	op	op CR.ofint <- ZqRing.ofint,	Structure map of $\mathbb{F}_q$: $0, 1, +, -, \cdot, (\cdot)^{-1}$, exponentiation, or integer embedding.
Rq.ec:85	op	op zroot = incoeff 426.	Primitive root used by the NTT formulas: $\zeta = [426]_q$.
Rq.ec:87	op	op br = BitEncoding.BitReverse.bsrev 8.	Eight-bit reversal map brevq.

Source	Kind	Exact EasyCrypt declaration line	Mathematical correspondence
Rq.ec:89	op	op ntt(p : poly) = Array256.init (fun i =>	Even/odd split transform over 128 terms; recorded as not the final HAETAE full NTT target.
Rq.ec:95	op	op invntt(p : poly) = Array256.init (fun i =>	Inverse even/odd split transform with factor 128^{-1} .
Rq.ec:106	op	op cmplx_mul (a :coeff * coeff, b : coeff * coeff, zzeta : coeff) : coeff * coeff =	Degree-one product modulo $X^2 - \zeta^e$: $(a_0 + a_1 X)(b_0 + b_1 X)$.
Rq.ec:111	op	op basemul(a b : poly) : poly = Array256.init (fun i =>	Pointwise base multiplication in the split NTT domain.

Hoare, phoare, and equiv Occurrences

Source	Kind	Exact EasyCrypt judgment line	Mathematical semantic reading
Fq.ec:194	hoare	hoare [M.__montgomery_reduce :	Hoare judgment for SREDC.
Fq.ec:304	phoare	phoare [M.__montgomery_reduce :	Probability-one Hoare judgment: the deterministic procedure terminates and satisfies the postcondition with probability 1.
Fq.ec:318	hoare	hoare [M.__fqmul :	Hoare judgment for Montgomery field multiplication.
Fq.ec:474	phoare	phoare [M.__fqmul :	Probability-one Hoare judgment: the deterministic procedure terminates and satisfies the postcondition with probability 1.
NTTEndToEnd.ec:34	hoare	hoare [NTT_Fq.NTT.ntt :	Hoare judgment that the imperative forward loop computes F .
NTTEndToEnd.ec:36	hoare	hoare [Hpoly_extract.M._poly_ntt :	Hoare judgment for the forward Jasmin entry point: decoded output is $F(p)$ with bounds.
NTTEndToEnd.ec:47	hoare	hoare [Hpoly_extract.M._poly_ntt :	Hoare judgment for the forward Jasmin entry point: decoded output is $F(p)$ with bounds.
NTTEndToEnd.ec:54	hoare	hoare [Hpoly_extract.M.poly_ntt_jazz :	Hoare judgment for the forward Jasmin entry point: decoded output is $F(p)$ with bounds.
NTTEndToEnd.ec:61	hoare	hoare [NTT_Fq.NTT.invntt :	Hoare judgment that the imperative inverse loop computes F^{-1} .
NTTEndToEnd.ec:63	hoare	hoare [Hpoly_extract.M._poly_invntt :	Hoare judgment for the inverse Jasmin entry point: decoded output is $R \cdot F^{-1}(p)$ with bounds.
NTTEndToEnd.ec:74	hoare	hoare [Hpoly_extract.M._poly_invntt :	Hoare judgment for the inverse Jasmin entry point: decoded output is $R \cdot F^{-1}(p)$ with bounds.
NTTEndToEnd.ec:81	hoare	hoare [Hpoly_extract.M.poly_invntt_jazz :	Hoare judgment for the inverse Jasmin entry point: decoded output is $R \cdot F^{-1}(p)$ with bounds.
NTTEndToEnd.ec:88	hoare	hoare [Hpoly_extract.M.poly_ntt_jazz :	Hoare judgment for the forward Jasmin entry point: decoded output is $F(p)$ with bounds.
NTTEndToEnd.ec:96	hoare	hoare [Hpoly_extract.M.poly_invntt_jazz :	Hoare judgment for the inverse Jasmin entry point: decoded output is $R \cdot F^{-1}(p)$ with bounds.
NTTFullAlgebra.ec:1736	hoare	hoare [NTT_Fq.NTT.ntt :	Hoare judgment that the imperative forward loop computes F .
NTTFullAlgebra.ec:3056	hoare	hoare [NTT_Fq.NTT.invntt :	Hoare judgment that the imperative inverse loop computes F^{-1} .
RefJasminNTT.ec:191	hoare	hoare [Hpoly_extract.M.__fqmul :	Hoare judgment for Montgomery field multiplication.
RefJasminNTT.ec:211	hoare	hoare [Hpoly_extract.M.__fqmul :	Hoare judgment for Montgomery field multiplication.
RefJasminNTT.ec:231	hoare	hoare [Hpoly_extract.M.__fqmul :	Hoare judgment for Montgomery field multiplication.
RefJasminNTT.ec:253	hoare	hoare [Hpoly_extract.M.__fqmul :	Hoare judgment for Montgomery field multiplication.
RefJasminNTT.ec:280	phoare	phoare [Hpoly_extract.M.__fqmul :	Probability-one Hoare judgment: the deterministic procedure terminates and satisfies the postcondition with probability 1.
RefJasminNTT.ec:589	hoare	hoare [Hpoly_extract.M.__fqmul :	Hoare judgment for Montgomery field multiplication.
RefJasminNTT.ec:621	hoare	hoare [Hpoly_extract.M.__fqmul :	Hoare judgment for Montgomery field multiplication.
RefJasminNTT.ec:642	phoare	phoare [Hpoly_extract.M.__fqmul :	Probability-one Hoare judgment: the deterministic procedure terminates and satisfies the postcondition with probability 1.
RefJasminNTT.ec:655	hoare	hoare [Hpoly_extract.M._fqmul :	Hoare judgment for Montgomery field multiplication.
RefJasminNTT.ec:688	phoare	phoare [Hpoly_extract.M._fqmul :	Probability-one Hoare judgment: the deterministic procedure terminates and satisfies the postcondition with probability 1.
RefJasminNTT.ec:1193	hoare	hoare [Hpoly_extract.M.__fqmul :	Hoare judgment for Montgomery field multiplication.
RefJasminNTT.ec:1216	phoare	phoare [Hpoly_extract.M.__fqmul :	Probability-one Hoare judgment: the deterministic procedure terminates and satisfies the postcondition with probability 1.
RefJasminNTT.ec:1675	hoare	hoare [Hpoly_extract.M.__fqmul :	Hoare judgment for Montgomery field multiplication.
RefJasminNTT.ec:1700	phoare	phoare [Hpoly_extract.M.__fqmul :	Probability-one Hoare judgment: the deterministic procedure terminates and satisfies the postcondition with probability 1.
RefJasminNTT.ec:1737	equiv	equiv poly_ntt_jazz_ref :	Relational program equality/refinement: related inputs lead to related outputs.
RefJasminNTT.ec:1746	equiv	equiv poly_invntt_jazz_ref :	Relational program equality/refinement: related inputs lead to related outputs.
RefJasminNTT.ec:1755	equiv	equiv poly_ntt_core_ref :	Relational program equality/refinement: related inputs lead to related outputs.
RefJasminNTT.ec:2083	equiv	equiv poly_invntt_core_ref :	Relational program equality/refinement: related inputs lead to related outputs.